



TASK

Back-End Web Development

Visit our website

Introduction

In this task, we will dive into back-end development. You will focus on server-side operations and the tools used to build robust web applications. You will learn about frameworks, databases, and API development, as well as gain crucial knowledge to succeed as a back-end developer. A comparison between front-end and back-end development will also be covered, highlighting their distinct roles and synergies. By the end of this task, you will be well-equipped to describe the key concepts and tools for web application development.

WHAT IS BACK-END WEB DEVELOPMENT?

Back-end development refers to building and maintaining the server-side components of a web or software application. It involves handling the logic, data storage, and information processing that enables an application's functionality. It can be thought of as the 'powerhouse' of a website. It consists of a server, an application, and a database.

Key aspects of back-end development (in no specific order) include server-side programming, web frameworks, application programming interfaces (APIs), databases, integration with third-party applications, security, testing and debugging, deployment of the website, and performance.

Back-end development ensures the seamless operation of web applications by managing the core functionality that users do not directly interact with but rely on for the application to work effectively. For instance, when you upload a photo to Instagram, the back-end ensures that the photo is stored securely in a database, links it to your account, and makes it available for your followers to view. Without robust back-end development, such processes would be prone to errors, leading to a poor user experience.

Imagine an online store: while the front end displays product images, the back end ensures accurate inventory counts, processes payments, and tracks orders. When you search for a product on Amazon, the back end fetches results from the database, filters them based on your preferences, and sends the data to the front end to display.

RESPONSIBILITIES OF A BACK-END DEVELOPER

A back-end developer is responsible for creating and managing the technology that enables the user interface of a website to function effectively.

Whenever you navigate to a website, such as <https://www.amazon.com/>, the site's servers send information to your computer or mobile device, which becomes the page you see. This is the result of the work of a back-end developer. In addition, when you enrolled in your bootcamp, the storage of your personal information was also due to the work of a back-end developer.

Back-end developers use server-side languages, which run on web servers, cloud-based servers, or a hybrid combination, to build an application. Examples of server-side languages are Node.js, PHP, Ruby, Python, Java, and .Net. Back-end developers also use tools like MongoDB, MySQL, Oracle, and SQL Server to find, save, or change data. The data is then served back to the user in the form of front-end code with which the user interacts. Anything you see on a website or an application is made possible by back-end programming, which exists on and is powered by a **server**. Back-end developers use the above-mentioned tools to create or contribute to web applications with well-documented, clean, and portable code.

Back-end developers are the engineers of a complex system. For instance, when building an online banking system, they ensure that account balances are updated instantly after transactions, applying rigorous security measures to protect sensitive information. Similarly, in a streaming service like Netflix, back-end developers manage data retrieval to ensure seamless playback, even with a weak internet connection.

A back-end developer takes completed front-end code and makes it functional using **dynamic** programming. For example, a back-end developer can make it possible for values to be populated into a drop-down menu by building the infrastructure that pulls values from the database.

The responsibilities of a back-end developer typically include:

- **Database management:** Designing, creating, and managing efficient database structures, optimising database queries, and ensuring data integrity and security. Think of a library system. The database holds information on books, members, and loans. A back-end developer ensures

that when a member borrows a book, the database updates to reflect this.

- **Performance optimisation:** Back-end developers optimise server-side code to enhance the website's performance, speed, and scalability, ensuring the site runs smoothly even with high traffic. For example, think of an e-commerce site during a holiday sale; a back-end developer optimises queries and server response times so users can browse and make purchases without experiencing delays.
- **Testing and debugging:** Debugging is an ongoing responsibility to identify and fix issues in the code, ensuring the system is error-free. Developers also write and execute tests to confirm that all features work as intended. In this case, a developer might simulate different user inputs to confirm that a login system correctly handles valid and invalid credentials.
- **Frameworks usage:** Staying up to date with trending frameworks and tools allows back-end developers to work efficiently and implement modern features. Frameworks like Django, Flask, or Express.js simplify complex programming tasks. A developer might use Django to rapidly build a web application for an educational platform with user accounts and content management systems.
- **Deployment:** Deploying a website involves integrating cloud computing services, managing server infrastructure, and ensuring the site runs smoothly once live. After building an app, the developer configures it to run on cloud platforms like AWS or Azure, handling server configurations and backups.
- **Server-side programming:** Back-end developers write server-side code in languages like Python, PHP, JavaScript, Ruby, and Node.js to handle requests and send responses. For a social media app, the back-end code ensures that when a user uploads a photo, it is stored on a server and appears correctly on their profile.
- **API integration:** Building and maintaining APIs allow systems to communicate and share data seamlessly. A developer might integrate a payment API like [Stripe](#), enabling secure online transactions for an e-commerce site.

- **Security:** Implementing strong security measures protects websites and applications from vulnerabilities like phishing and hacking. Back-end developers might use encryption to protect user passwords and implement two-factor authentication for sensitive actions.
- **Collaboration with front-end developers:** Back-end developers work closely with front-end developers to ensure the visual and functional components of a website/application align seamlessly. A back-end developer collaborates with a front-end team to make sure that user-submitted reviews appear correctly on a product page.
- **Version control:** Using tools like Git allows developers to manage code changes, collaborate with team members, and maintain a well-documented codebase. When multiple developers are working on an application, version control prevents conflicts and ensures smooth integration of features.

By combining these skills, back-end developers provide the foundation that supports modern, dynamic, and scalable web applications.

BACK-END DEVELOPER TOOLS

Programming languages

Back-end development relies on a variety of programming languages, each chosen to meet the specific demands of a project. These languages are not just tools; they shape how efficiently and effectively a system operates.

- **Python:** Valued for its readability and vast ecosystem, Python supports frameworks like Flask and Django, making it ideal for building scalable, data-driven applications.
- **Java:** A cornerstone of enterprise systems, Java powers large-scale, robust applications using frameworks such as Spring. It excels in handling complex, high-performance systems.
- **JavaScript:** While often associated with front-end development, JavaScript's Node.js runtime makes it a powerful choice for creating fast, asynchronous server-side applications.
- **Ruby:** Known for its simplicity and speed of development, Ruby, coupled with the Ruby on Rails framework, is favoured for startups aiming to launch quickly.

- **PHP:** A workhorse of the web, PHP drives platforms like WordPress, offering straightforward solutions for dynamic content management and small to medium-scale web applications.

These languages define the foundation of back-end development, enabling the creation of secure, scalable, and dynamic systems tailored to various business needs.

Frameworks

There are two types of frameworks used by developers. One is a testing framework, and the other is a development framework. Some commonly used development frameworks are outlined below.

- **Flask:** A lightweight and flexible Python web framework, perfect for building simple to medium-sized web applications. It is favoured by developers for its minimalistic approach, requiring little boilerplate code to start a project. Flask offers modularity, allowing developers to add only the components they need, making it easy to customise and extend. Its simplicity makes it easy to learn and integrate with other tools, while still providing robust support for various web development tasks.
- **Django:** A full-stack Python framework that emphasises rapid development and clean, pragmatic design. It comes with a range of built-in features, such as URL routing, authentication, an advanced ORM, and an admin interface for managing databases. Django allows developers to build secure and scalable web applications efficiently. Its robust ecosystem and extensive documentation make it a popular choice for both beginners and experienced developers.
- **Express.js:** A minimalistic yet powerful framework for building web applications and APIs with Node.js. It streamlines routing, middleware handling, and server-side operations, providing flexibility in managing requests and responses. Express is particularly suited for developing real-time applications and handling asynchronous tasks. With its extensive plugin ecosystem, developers can easily extend its functionality according to their project needs.
- **Laravel:** A PHP framework known for its elegant and expressive syntax, which simplifies complex web development tasks. It comes with a robust ORM (Eloquent), built-in authentication, and features like routing, caching, and queue management. Laravel promotes best practices in web development, ensuring security, performance, and scalability. Its rich library of tools and community support makes it a popular choice for web developers.

Some commonly used testing frameworks are outlined below.

- **Selenium:** An open-source framework widely used for web testing. It supports functional and regression testing by automating web browsers, allowing developers to create tests that simulate real user interactions with web applications. Selenium is highly flexible and works with multiple programming languages, including Python, Java, and JavaScript.
- **Pytest:** A Python testing framework known for its simplicity and scalability. It provides an easy way to write unit tests, functional tests, and integration tests with a vast collection of plugins to extend its functionality. Pytest supports parameterised testing, fixtures, and a wide range of assertions, making it a versatile tool for testing Python applications.
- **PHPUnit:** The standard testing framework for PHP, providing a robust way to write and execute unit tests. It helps in ensuring the correctness of code by catching bugs early in the development process. PHPUnit is compatible with various PHP versions and integrates seamlessly with continuous integration pipelines for automated testing.
- **Mocha:** A JavaScript testing framework commonly used in web development for writing unit and integration tests. It supports both asynchronous and synchronous testing, making it ideal for testing JavaScript and Node.js applications. Mocha's simple interface, combined with its ability to organise test suites effectively, ensures a smooth testing experience.
- **Chai:** An assertion library for JavaScript, often used alongside Mocha to create clear and descriptive tests. It allows developers to write expressive, easy-to-read assertions, validating the expected behaviour of code with detailed feedback on test results. Chai's flexibility makes it suitable for a wide range of testing scenarios.

Databases

The choice of a database depends on the nature of the application and considerations such as performance, scalability, and storage required. The following are some of the most common databases developers use:

Relational databases:

- **MySQL:** An open-source relational database widely used for web applications due to its reliability and extensive features. MySQL supports

advanced querying and integrates well with frameworks like Laravel and Django.

- **Oracle:** A robust enterprise-level relational database that provides high performance, security, and scalability, making it ideal for business-critical applications.
- **PostgreSQL:** An open-source relational database with a strong focus on standards-compliance, extensibility, and complex data types, ideal for applications requiring advanced data management.
- **Microsoft SQL server:** A powerful relational database designed for enterprise applications, offering features like transaction management, high availability, and integration with various Microsoft services.

NoSQL databases:

- **MongoDB:** A widely-used NoSQL database optimised for handling unstructured or semi-structured data, such as content management systems and analytics platforms.
- **Cassandra:** A distributed NoSQL database that offers high scalability and availability, ideal for applications requiring real-time data handling and fault-tolerance.
- **Redis:** An in-memory key-value store used for caching, session management, and real-time data processing, providing low-latency access to data.

Time-series databases:

- **InfluxDB:** A time-series database designed for monitoring and analytics applications. It efficiently stores and queries time-stamped data, making it suitable for IoT, logs, and sensor data management.

API development tools

Postman is one of the most widely used tools for API development and testing. It provides a comprehensive platform for designing, testing, and managing APIs. With its user-friendly interface, Postman allows developers to create requests, simulate responses, and debug APIs efficiently. APIs act as bridges between the front end and back end, enabling smooth data exchange and communication between different systems.

Purpose in back-end development

In back-end development, Postman, for example, plays a crucial role by streamlining the API development process. Such a tool provides a structured environment where developers can:

- Design API endpoints by sending different types of requests (GET, POST, PUT, DELETE) to servers.
- Validate responses to ensure APIs return the expected data and perform correctly.
- Collaborate on API documentation, making it easier for teams to understand API functionality and integrate new features.
- Automate testing through collections, which allow for repeated testing of APIs across different environments, improving the reliability and stability of back-end services.

Servers

There are two main types of servers which will be elaborated on in the next section.

1. Web servers
2. Application servers

These tools are used to create interactive, responsive, and dynamic websites.

SERVERS

To be a full-stack web developer, one needs to know what is happening on the server at the back end of a web application. As mentioned, web servers and application servers are used for the majority of web applications.

Web servers

A web server is a computer with an internet connection and the necessary software stored on it to make web pages available to clients. A web server will store resources associated with web applications, for example, any images, HTML, CSS, and JavaScript files needed to make a web application function. In addition, a web server must have software installed on it that allows it to act as an HyperText Transport Protocol (HTTP) server. An HTTP server (e.g., IIS) is a component that receives HTTP requests and sends the appropriate HTTP responses.

Web browsers use HTTP to communicate with web servers. An HTTP request is sent from your browser to the target web server whenever you click a link on a web page, submit a form, or run a search. Web servers wait for these request messages and then process them when they arrive. The web servers reply to the web browser with an HTTP response message.

A static website returns the same hard-coded content from the web server whenever a particular resource is requested.

Application servers

Application servers contain the business logic (code) needed to build resources to be dynamically passed back to the browser. An application server will usually have a web framework running on it. For example, a Node application server will usually use Express. The framework can match requests and dynamically generate responses.

Static and Dynamic Applications

If you only want your website to serve static pages, you **only** need a web server as shown in the diagram below.

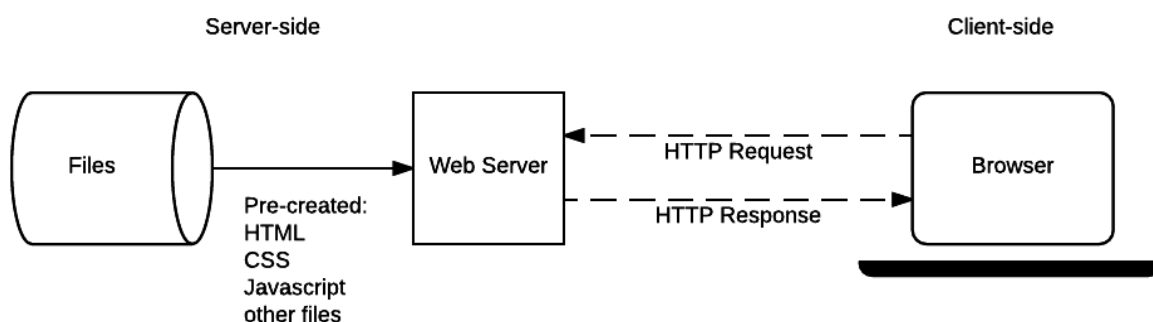


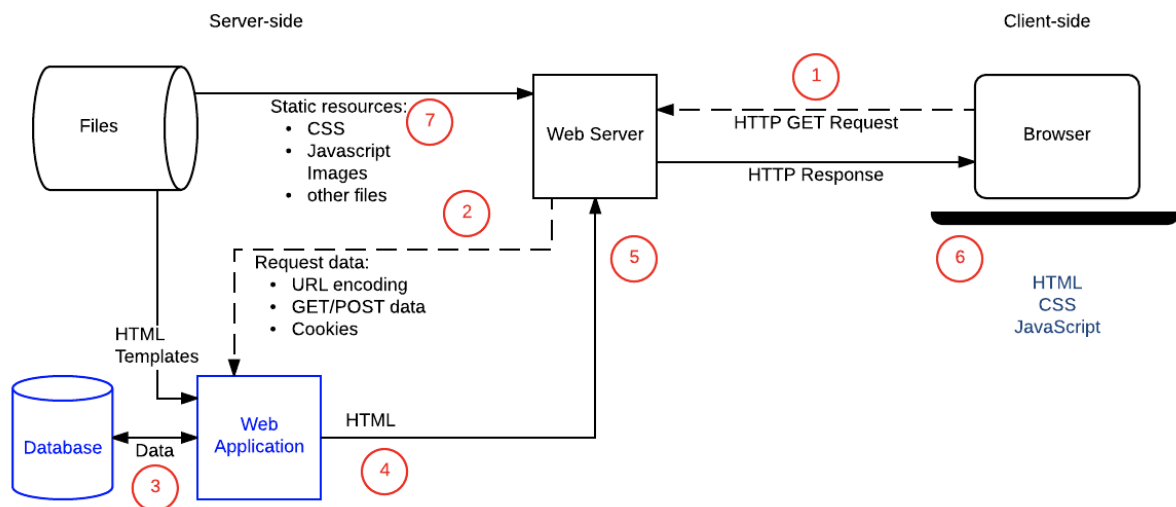
Image source:

https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Client-Server_overview

This image above demonstrates that for static applications the website only serves pre-created HTML, CSS, JavaScript, and other files. The diagram shows that a web server is the only server-side component needed. The client side is simply a browser that sends HTTP requests to the web server and receives the static responses.

However, if you want to build a web application that responds to requests with

dynamic resources, you need a web server and an application server. Review the diagram below to identify the additional elements in a dynamic web application.



Web Server Architecture for a Dynamic Site

Image source:

https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Introduction

The image above depicts the web server architecture for a dynamic web application, which differs from the setup for a static website.

In the dynamic architecture, the server-side components include not just the web server, but also a separate web application server. The client's browser sends an HTTP GET request to the web server, which then forwards any non-static requests to the web application server.

The web application server is responsible for handling the dynamic aspects of the web application. It can fetch data from a database, process it, and generate the appropriate HTML, CSS, and JavaScript content to be returned to the client. The web server then sends this dynamically generated response back to the client's browser.

Compared to the static website architecture, the dynamic setup requires an additional web application server to handle the more complex, data-driven functionality of the web application. Typically, a reverse proxy sits in front of both the web server and application server, routing traffic to the appropriate server based on the request.

Typically a reverse proxy sits in front of your web and application servers and routes traffic to the appropriate server. An application server is most likely run as a web

server proxy. The web server usually just passes non-static requests to the application server.



Take note:

Here are a few **helpful definitions** from [NGINX](#):

“A **proxy server** is a go-between or intermediary server that forwards requests for content from multiple clients to different servers across the Internet.”

“A **reverse proxy server** is a proxy server that typically sits behind the firewall in a private network and directs client requests to the appropriate back-end server. A reverse proxy provides additional abstraction and control to ensure the smooth flow of network traffic between clients and servers.”

FRONT-END DEVELOPER VS A BACK-END DEVELOPER

The following table offers a comparative overview of front-end and back-end developers, highlighting their distinct roles and responsibilities in web development:

Aspect	Front-end developer	Back-end developer
Programming languages	HTML, CSS, and JavaScript	Python, Ruby, Java, PHP, C#, JavaScript.
Focus	User interface (UI) and user experience (UX)	Server-side logic and processing
Frameworks	React, Angular, etc.	Django, Express.js, Flask etc.
Accessibility	Ensures the application is accessible to everyone	Concerned with performance, scalability, and speed
Testing	Conducts UI and cross-border testing	Conducts unit testing, integration testing, and back-end testing
Search engine optimisation (SEO)	Implements best SEO practices	Less involved in SEO considerations

Aspect	Front-end developer	Back-end developer
Programming languages	HTML, CSS, and JavaScript	Python, Ruby, Java, PHP, C#, JavaScript.
Focus	User interface (UI) and user experience (UX)	Server-side logic and processing
Frameworks	React, Angular, etc.	Django, Express.js, Flask etc.
Accessibility	Ensures the application is accessible to everyone	Concerned with performance, scalability, and speed
Testing	Conducts UI and cross-border testing	Conducts unit testing, integration testing, and back-end testing
practices		
Performance	Optimises page load speeds	Optimises back-end performances

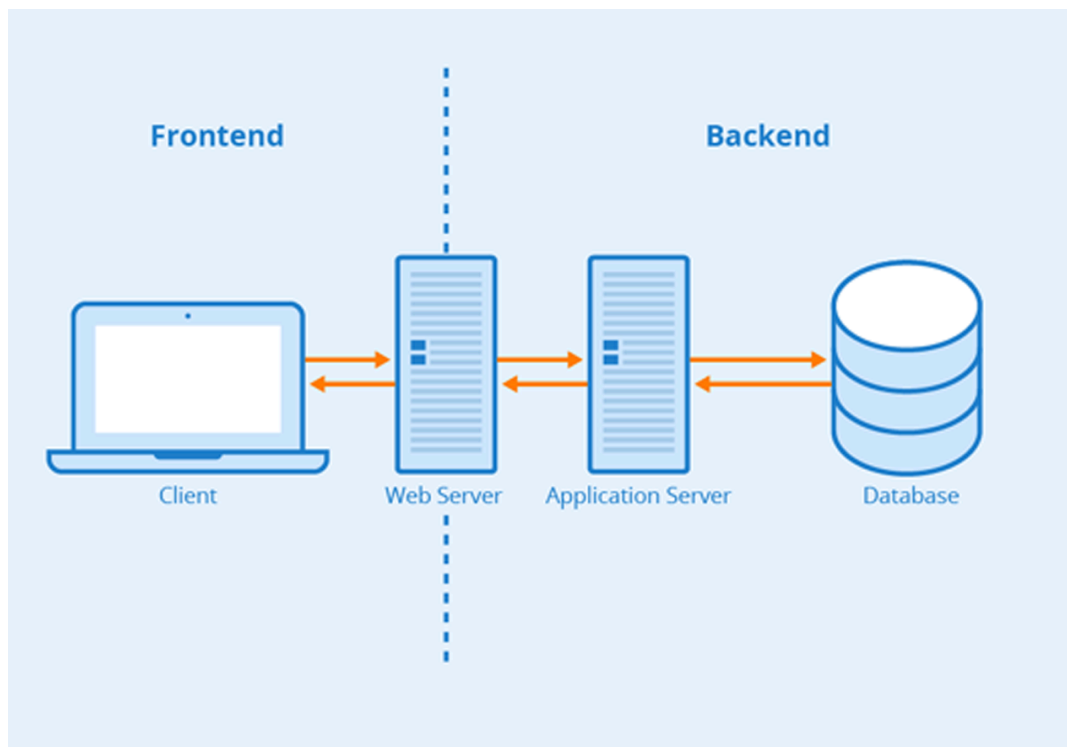


Image source: <http://www.totalvdo.com/back-end-development.html>

The increasing demand for sophisticated web applications makes back-end development a valuable skill to learn. Back-end development enables the functionality, reliability, and performance of web applications ensuring a smooth

user experience. In addition, the continuous evolution of technology makes back-end development a rewarding field.



Practical task 1

Use what you have learnt in this task, supplemented with some self-study, to **briefly** answer the questions listed below. Leverage keywords from the question to get nuanced answers from Google or your search engine of choice. Submit your answers in a PDF document entitled **BackendWebDev**.

1. What is the primary focus of back-end web development?
2. Why is understanding databases important for back-end developers?
3. What role do APIs play in back-end development?
4. What are some commonly used server-side languages for back-end development?
5. Explain the importance of security in back-end development.
6. How do frameworks like Django and Flask simplify back-end development?

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
